

รายงานปฏิบัติการ Lab 10

Spring WebFlux และ WebClient

CP353002 หลักการออกแบบและพัฒนาซอฟต์แวร์

ชื่อ	Supawat Khaithong
รหัสนักศึกษา	673380001-2
Section	1
หัวข้อ	Reactive Programming ด้วย Spring WebFlux

รายงานนี้แสดงแนวคิด การเขียนโค้ด และผลการทดสอบ REST API แบบ non-blocking

1 บทนำ

งาน Lab นี้เป็นการทดลองเขียน REST API ด้วย Spring WebFlux โดยข้อมูลจะถูกส่งออกมาในรูปแบบ Mono และ Flux แทนการคืนค่าแบบปกติ จุดสำคัญคือโปรแกรมไม่ต้องหยุดรอ I O ทุกครั้ง ทำให้ thread เดิมสามารถไปรับงานอื่นต่อได้ เหมาะกับระบบที่มี request เข้ามาพร้อมกันจำนวนมาก

ในโปรเจกต์ได้ทำ API สำหรับดึงข้อมูลสินค้า เพิ่มสินค้า ลบสินค้า กรองตามหมวดหมู่ และคำนวณราคาหลังส่วนลด รวมถึงเขียน ProductWebClient สำหรับเรียก API แบบ reactive

2 Reactive Programming และ Blocking

Blocking I O เป็นการทำงานที่ thread ต้องรอจนกว่างานเดิมจะเสร็จ เช่น รอ database หรือรอ HTTP response ระหว่างที่รอ thread นั้นจะยังไม่สามารถรับงานอื่นได้ ถ้ามีผู้ใช้พร้อมกันมากก็ต้องใช้ thread มากขึ้น

Reactive Programming จะมองข้อมูลเป็น stream และทำงานแบบ asynchronous เมื่อส่ง request ออกไป thread ไม่จำเป็นต้องยืนรอ แต่สามารถไปทำงานอื่นก่อน แล้วกลับมาประมวลผลเมื่อข้อมูลมาถึง วิธีนี้ช่วยให้รองรับงานที่มีการรอ I O จำนวนมากได้ดีขึ้น

หัวข้อ	Blocking	Reactive
การรอ	thread รอผลลัพธ์	thread ไปทำงานอื่นได้
รูปแบบข้อมูล	คืนค่าปกติ	Publisher ส่งข้อมูลเป็น stream
การรองรับผู้ใช้	ต้องเพิ่ม thread ตามงาน	ใช้ thread จำนวนน้อยรับหลายงาน
งานที่เหมาะสม	งานทั่วไปที่ไม่ซับซ้อน	งาน HTTP streaming และ concurrent สูง

3 Mono และ Flux

Mono ใช้แทนผลลัพธ์ที่มีได้ 0 หรือ 1 ค่า เช่น การค้นหาสินค้าด้วย id การบันทึกสินค้า และการคำนวณราคา ส่วน Flux ใช้แทนผลลัพธ์ที่มีได้ 0 ถึงหลายค่า เช่น การดึงสินค้าทั้งหมดหรือการกรองสินค้าตาม category

ชนิด	จำนวนข้อมูล	ตัวอย่างในงาน
Mono<Product>	0 หรือ 1 รายการ	findById และ save
Mono<Double>	0 หรือ 1 ค่า	getDiscountedPrice
Mono<Void>	ไม่มีข้อมูลตอบกลับ	deleteById
Flux<Product>	0 ถึงหลายรายการ	findAll และ findByCategory

เหตุผลที่เลือกใช้สองชนิดนี้คือให้ชนิดของผลลัพธ์สื่อจำนวนข้อมูลได้ชัดเจน ถ้าคาดว่าจะมีสินค้าเพียงตัวเดียวจะใช้ Mono แต่ถ้าเป็นรายการจะใช้ Flux

4 ลำดับการทำงานของโปรแกรม

เมื่อ client เรียก endpoint คำขอจะเข้า ProductController จากนั้น Controller จะเรียก ProductService เพื่อทำ business logic และ Service จะเรียก ProductRepository เพื่ออ่านหรือแก้ไขข้อมูลใน ConcurrentHashMap ผลลัพธ์จะถูกส่งย้อนกลับเป็น Mono หรือ Flux โดยไม่เรียก block

```
Client request
-> ProductController
   -> ProductService
       -> ProductRepository
         <- Mono หรือ Flux
         <- JSON response
```

5 การเขียน ProductRepository

Repository เป็นชั้นที่เก็บข้อมูลตัวอย่างใน ConcurrentHashMap แล้วแปลงค่าที่ได้ให้เป็น reactive type โค้ด findById ใช้ Mono.justOrEmpty เพราะถ้าไม่พบ id จะได้ Mono.empty โดยอัตโนมัติ ส่วน findAll ใช้ Flux.fromIterable เพื่อส่งสินค้าทีละรายการ

```
public Mono<Product> findById(String id) {
    return Mono.justOrEmpty(store.get(id));
}

public Flux<Product> findAll() {
    return Flux.fromIterable(store.values());
}

public Mono<Product> save(Product product) {
    store.put(product.getId(), product);
    return Mono.just(product);
}

public Mono<Void> deleteById(String id) {
    return Mono.fromRunnable(() -> store.remove(id));
}

public Flux<Product> findByCategory(String category) {
    return findAll()
        .filter(product -> product.getCategory() != null
            && product.getCategory().equalsIgnoreCase(category));
}
```

จุดที่ใช้ operator ชัดเจนคือ filter ซึ่งตรวจสอบสินค้าแต่ละตัวใน stream แล้วปล่อยผ่านเฉพาะตัวที่ category ตรงกับค่าที่รับเข้ามา

6 การเขียน ProductService

Service ทำหน้าที่เชื่อม Controller กับ Repository และใส่ business logic เพิ่มเติม ถ้าค้นหา id แล้วไม่พบจะเปลี่ยน Mono.empty ให้เป็น error ด้วย switchIfEmpty ในการบันทึกสินค้า ถ้าไม่มี id จะสร้าง UUID ให้ก่อน ส่วนราคาหลังลดใช้ map แปลง Product ให้เป็น Double

```

public Mono<Product> getById(String id) {
    return repository.findById(id)
        .switchIfEmpty(Mono.error(
            new RuntimeException("Product not found: " + id)));
}

public Mono<Product> save(Product product) {
    if (product.getId() == null || product.getId().isBlank()) {
        product.setId(UUID.randomUUID().toString());
    }
    return repository.save(product);
}

public Mono<Double> getDiscountedPrice(String id) {
    return getById(id).map(Product::getDiscountedPrice);
}

```

map เหมาะกับจุดนี้เพราะการคำนวณ getDiscountedPrice เป็นการแปลงค่าปกติจาก Product เป็น Double และไม่ได้อื่น Publisher ตัวใหม่

7 การเขียน ProductController

Controller กำหนด URL และ HTTP method ด้วย annotation แล้วส่งงานต่อไปยัง Service โดยคืน Mono หรือ Flux ออกไปตรง ๆ Spring WebFlux จะ subscribe และแปลงผลลัพธ์เป็น JSON response ให้เอง

```

@GetMapping
public Flux<Product> getAll() {
    return service.getAll();
}

@PostMapping
public Mono<Product> save(@RequestBody Product product) {
    return service.save(product);
}

@DeleteMapping("/{id}")
public Mono<Void> delete(@PathVariable String id) {
    return service.delete(id);
}

@GetMapping("/category/{category}")
public Flux<Product> getByCategory(@PathVariable String category) {
    return service.getByCategory(category);
}

@GetMapping("/{id}/price")
public Mono<Double> getDiscountedPrice(@PathVariable String id) {
    return service.getDiscountedPrice(id);
}

```

8 การใช้ WebClient

WebClient ใช้เรียก HTTP API แบบ non-blocking โดย method chain แต่ละส่วนมีหน้าที่ต่อกันดังนี้

- get post และ delete ใช้เลือก HTTP method ที่ต้องการส่ง
- uri ใช้กำหนด path และแทนค่า path variable เช่น id หรือ category
- retrieve ใช้รับ response จาก server และเข้าสู่ขั้นตอนอ่าน body
- bodyToMono ใช้เมื่อ response มีค่าเดียว ส่วน bodyToFlux ใช้เมื่อ response เป็นหลายรายการ

```

public Flux<Product> getAllProducts() {
    return client.get()
        .uri("/products")
        .retrieve()
        .bodyToFlux(Product.class);
}

public Mono<Product> createProduct(Product product) {
    return client.post()
        .uri("/products")
        .bodyValue(product)
        .retrieve()
        .bodyToMono(Product.class);
}

```

9 สรุป API ที่พัฒนา

Method	URL	Return	หน้าที่
GET	/products/{id}	Mono<Product>	ค้นหาสินค้าจาก id
GET	/products	Flux<Product>	ดึงสินค้าทั้งหมด
POST	/products	Mono<Product>	เพิ่มสินค้าใหม่
DELETE	/products/{id}	Mono<Void>	ลบสินค้า
GET	/products/category/{category}	Flux<Product>	กรองตามหมวดหมู่
GET	/products/{id}/price	Mono<Double>	หาราคาหลังลด

10 ผลการทดสอบ API

1 ดึงสินค้าตามรหัส

คำสั่งทดสอบ: GET <http://localhost:8081/products/1>

The screenshot shows a REST client interface with the following details:

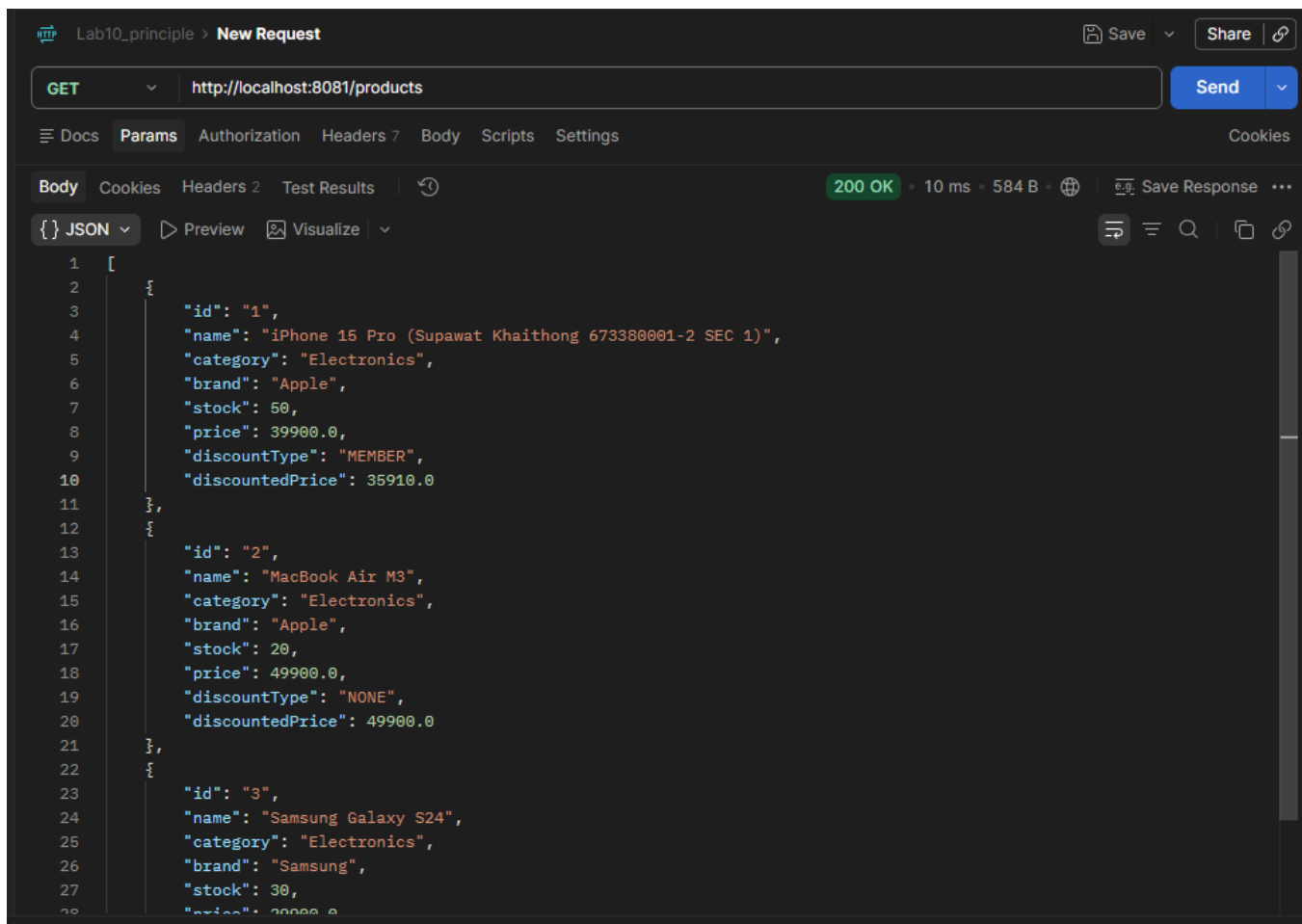
- Request:** Method: GET, URL: `http://localhost:8081/products/1`
- Response:** Status: 200 OK, Time: 4 ms, Size: 263 B
- Body (JSON):**

```
{  "id": "1",  "name": "iPhone 15 Pro (Supawat Khaithong 673380001-2 SEC 1)",  "category": "Electronics",  "brand": "Apple",  "stock": 50,  "price": 39900.0,  "discountType": "MEMBER",  "discountedPrice": 35910.0}
```

รูปที่ 1 ผลการทดสอบ ดึงสินค้าตามรหัส

2 ดึงสินค้าทั้งหมด

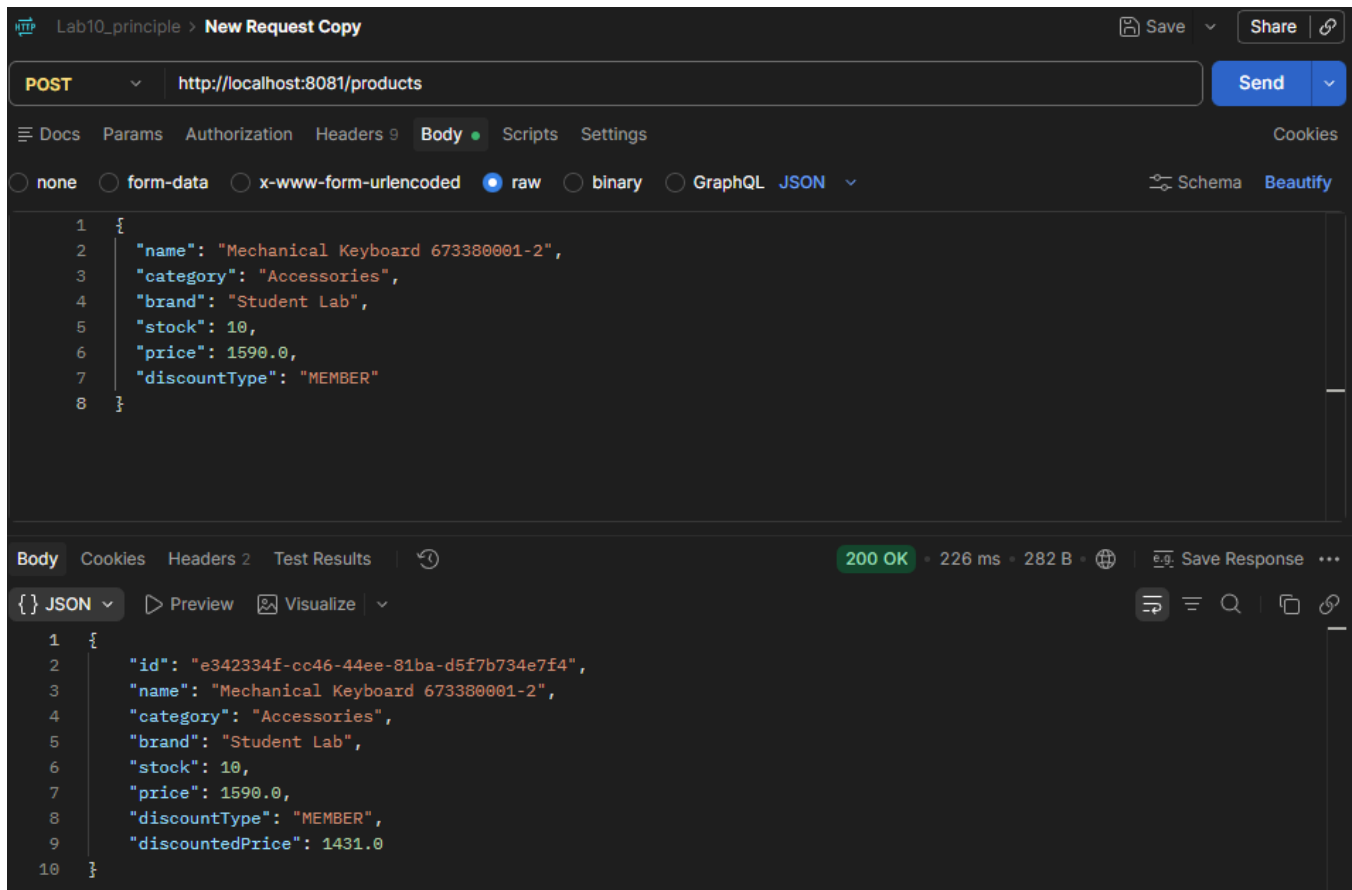
คำสั่งทดสอบ: GET <http://localhost:8081/products>



รูปที่ 2 ผลการทดสอบ ดึงสินค้าทั้งหมด

3 เพิ่มสินค้าใหม่

คำสั่งทดสอบ: POST <http://localhost:8081/products>



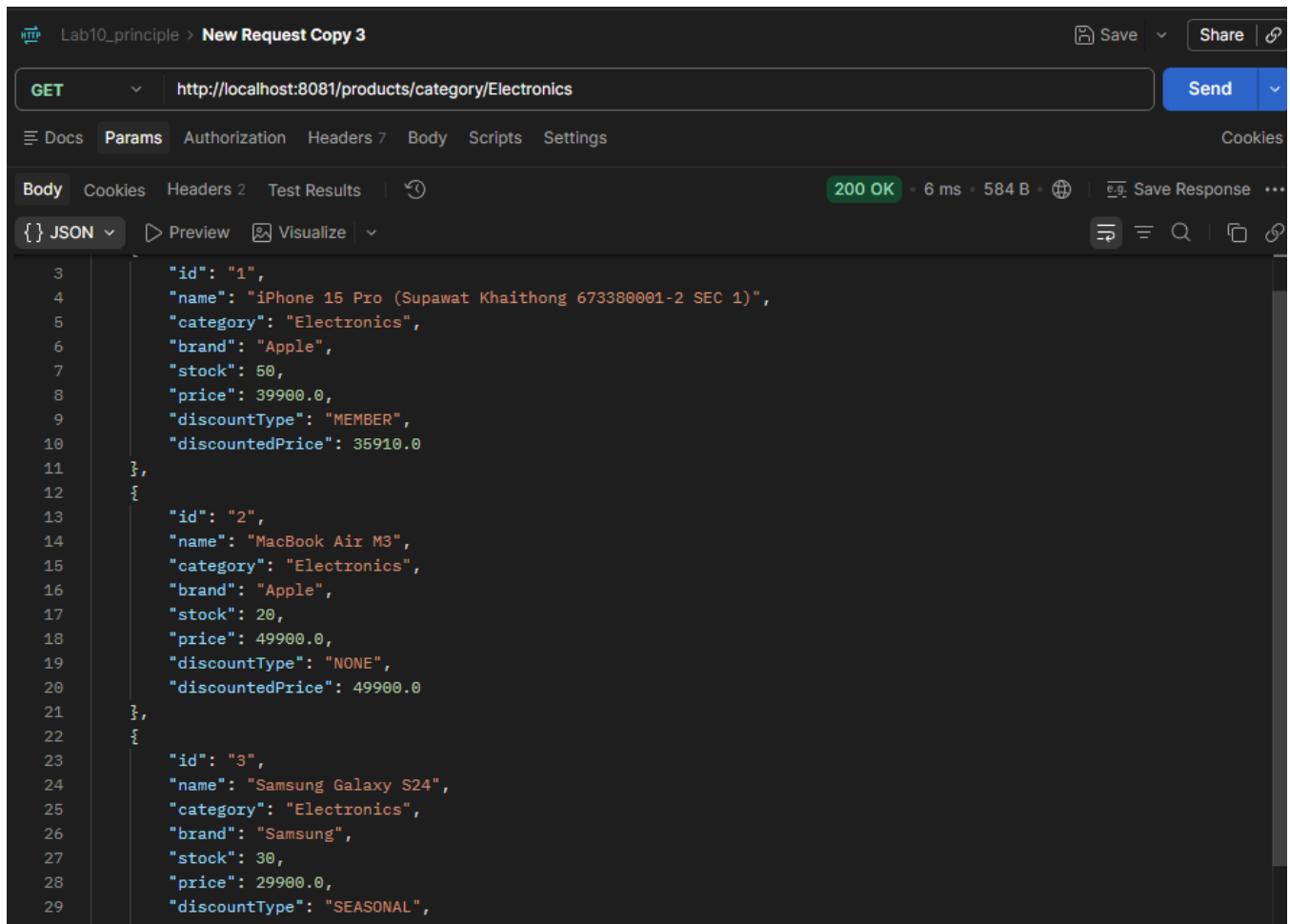
รูปที่ 3 ผลการทดสอบ เพิ่มสินค้าใหม่

ตัวอย่าง JSON สำหรับ POST

```
{
  "name": "Mechanical Keyboard 673380001-2",
  "category": "Accessories",
  "brand": "Student Lab",
  "stock": 10,
  "price": 1590.0,
  "discountType": "MEMBER"
}
```

4 กรองสินค้าตามหมวดหมู่

คำสั่งทดสอบ: GET <http://localhost:8081/products/category/Electronics>



```
Lab10_principle > New Request Copy 3
GET http://localhost:8081/products/category/Electronics
Send

Body Cookies Headers 2 Test Results
200 OK • 6 ms • 584 B • Save Response

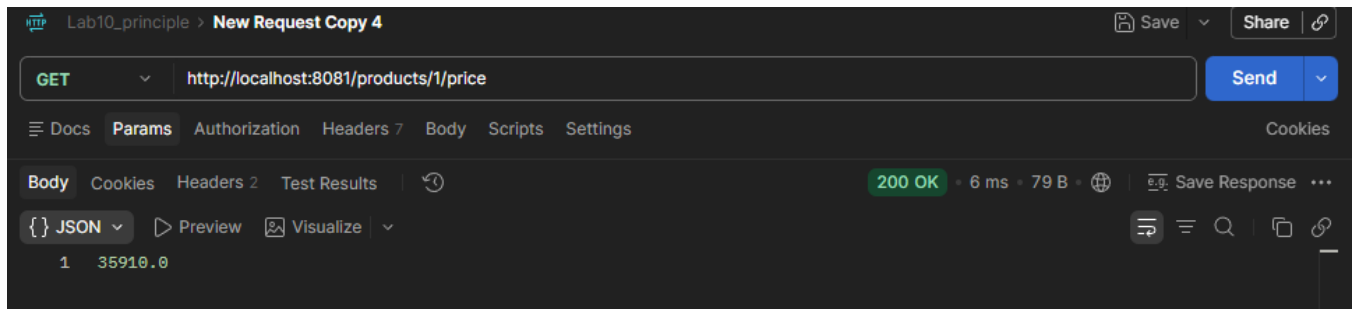
[JSON] Preview Visualize

[
  {
    "id": "1",
    "name": "iPhone 15 Pro (Supawat Khaithong 673380001-2 SEC 1)",
    "category": "Electronics",
    "brand": "Apple",
    "stock": 50,
    "price": 39900.0,
    "discountType": "MEMBER",
    "discountedPrice": 35910.0
  },
  {
    "id": "2",
    "name": "MacBook Air M3",
    "category": "Electronics",
    "brand": "Apple",
    "stock": 20,
    "price": 49900.0,
    "discountType": "NONE",
    "discountedPrice": 49900.0
  },
  {
    "id": "3",
    "name": "Samsung Galaxy S24",
    "category": "Electronics",
    "brand": "Samsung",
    "stock": 30,
    "price": 29900.0,
    "discountType": "SEASONAL",
    "discountedPrice": 29900.0
  }
]
```

รูปที่ 4 ผลการทดสอบ กรองสินค้าตามหมวดหมู่

5 คำนวณราคาหลังส่วนลด

คำสั่งทดสอบ: GET <http://localhost:8081/products/1/price>



รูปที่ 5 ผลการทดสอบ คำนวณราคาหลังส่วนลด

6 ลบสินค้า

คำสั่งทดสอบ: DELETE http://localhost:8081/products/{id}

The screenshot shows a REST client interface with the following details:

- Request Method:** DELETE
- Request URL:** http://localhost:8081/products/1
- Response Status:** 200 OK
- Response Time:** 9 ms
- Response Size:** 40 B
- Response Content Type:** application/json
- Response Body:** Raw (Preview)

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

รูปที่ 6 ผลการทดสอบ ลบสินค้า

11 ผลการทดสอบด้วย JUnit

ใช้ StepVerifier ตรวจสอบ signal ของ Mono และ Flux โดยทดสอบการค้นหาแบบพบและไม่พบข้อมูล การดึงทั้งหมด การบันทึก และการกรองตาม category ผลการรันล่าสุดผ่านทั้งหมด 6 tests ไม่มี failure และไม่มี error

```
Tests run: 6, Failures: 0, Errors: 0, Skipped: 0
```

12 สรุปผล

จากการทดลองสามารถสร้าง REST API ด้วย Spring WebFlux ได้ครบตามโจทย์ และเข้าใจการเลือกใช้ Mono กับ Flux มากขึ้น Repository ส่งข้อมูลเป็น reactive stream ส่วน Service ใช้ operator เช่น map filter และ switchIfEmpty เพื่อจัดการข้อมูลก่อนส่งไปยัง Controller นอกจากนี้ WebClient ยังช่วยให้เรียก API ต่อเป็น method chain ได้โดยไม่ต้อง block thread

สิ่งที่ได้เห็นชัดจากงานนี้คือ reactive ไม่ได้ทำให้โค้ดทุกอย่างเร็วขึ้นทันที แต่ช่วยให้ระบบใช้ thread ได้คุ้มค่าขึ้นเมื่อมีงาน I O และผู้ใช้จำนวนมากพร้อมกัน

เอกสารอ้างอิง

- Spring Framework Documentation. WebClient.
- Project Reactor Reference Guide. Introduction to Reactive Programming.
- Baeldung. Guide to Spring WebFlux.